

EasyMagpieTTS to vLLM-Omni/Triton Port Report

Codex

2026-06-24

EasyMagpieTTS to vLLM-Omni/Triton Port Report

Summary

This report documents the work completed to continue and validate the EasyMagpieTTS to vLLM-Omni/Triton port in:

`/home/siddhartht/tts/speechLM/NeMoDuplexRealtime`

The main outcome was a working local Triton deployment path for the converted EasyMagpieTTS model. The codec TensorRT plan was built, a default English speaker embedding was baked, Triton loaded both models successfully, and a whole-text end-to-end request returned audio.

Starting Point

The previous session had already completed the model and codec conversion stages:

- The EasyMagpie .nemo checkpoint had been converted to a vLLM-compatible model directory.
- The codec decoder had been exported to ONNX.
- Conversion and ONNX export scripts had been patched for local-checkout execution and lazy NeMo imports.
- EasyMagpie checkpoint restore had been patched to honor CPU `map_location`.
- vLLM plugin import and parity/unit tests had passed.

The remaining missing pieces were:

- A TensorRT codec plan at `examples/tts/easymagpie_vllm_omni/model_repository/codec/1/model.plan`.
- A baked default speaker embedding at `examples/tts/easymagpie_vllm_omni/easymp_vllm_model/speaker_embeddings/eng.pt`.
- Triton startup validation.

- An end-to-end request or benchmark.

Artifact Layout

Source model artifacts:

```
/home/siddhartht/tts/speechLM/NeMo_main/models/easy_tts
```

Important generated/runtime artifacts:

```
examples/tts/easymagpie_vllm_omni/easymp_vllm_model/  
examples/tts/easymagpie_vllm_omni/codec.onnx  
examples/tts/easymagpie_vllm_omni/model_repository/codec/1/model.plan  
examples/tts/easymagpie_vllm_omni/easymp_vllm_model/speaker_embeddings  
/eng.pt
```

Docker Runtime Setup

I first verified that Docker and the GPU were available. The local host had an NVIDIA RTX A4500 with about 20 GB VRAM. Docker access was available after the environment changed to unrestricted execution.

I pulled the Triton base image used by the example Dockerfile:

```
docker pull nvcr.io/nvidia/tritonserver:26.02-py3
```

Then I inspected the base image. It provided:

- Triton Server 2.66.0.
- TensorRT 10.15.1 trtexec.
- GPU visibility through the NVIDIA runtime.

The base image did not include the vLLM/vLLM-Omni Python dependencies, so I built the project-specific runtime image:

```
docker build --network=host -t easymp-vllm-omni  
examples/tts/easymagpie_vllm_omni/
```

The build installed:

- vllm==0.21.0
- vllm_omni==0.21.0rc1
- the EasyMagpie vLLM-Omni plugin package
- ONNX and Triton client utilities needed by the export and benchmark scripts

I then validated imports inside the image:

```
torch 2.11.0+cu130  
vllm 0.21.0  
vllm_omni 0.21.0rc1  
EasyMagpie plugin registered successfully
```

Building the TensorRT Codec Plan

The exported ONNX codec decoder was already present:

```
examples/tts/easymagpie_vllm_omni/codec.onnx
```

I built the TensorRT plan inside the easymp-vllm-omni container:

```
docker run --rm --gpus all -v "$PWD":/workspace -w /workspace easymp-
vllm-omni \
  python3
examples/tts/easymagpie_vllm_omni/scripts/export_codec_decoder_trt.py \
  --onnx-path examples/tts/easymagpie_vllm_omni/codec.onnx \
  --trt-path
examples/tts/easymagpie_vllm_omni/model_repository/codec/1/model.plan \
  --batch-profile 1 8 32 \
  --frames-profile 15 15 15 \
  --fp32
```

TensorRT parsed the ONNX successfully, inferred 16 stacked quantizer/codebook channels, generated the engine, and ran its built-in inference check. The generated plan was:

```
examples/tts/easymagpie_vllm_omni/model_repository/codec/1/model.plan
```

The plan size was about 210 MB.

First Triton Startup Attempt

I started Triton using the generated model repository:

```
docker run --rm --detach --gpus all --shm-size=8g \
  -p 18000:8000 -p 18001:8001 -p 18002:8002 \
  -v "$PWD":/workspace -w /workspace \
  --name easymp-triton-codex \
  easymp-vllm-omni \
  tritonserver
--model-repository=examples/tts/easymagpie_vllm_omni/model_repository
```

The codec TensorRT model loaded successfully. The EasyMagpie Python backend also initialized vLLM-Omni successfully after the first-run Torch compile and CUDA graph capture steps. Both models reached READY:

```
codec    READY
easyp    READY
```

The Triton health endpoint returned HTTP 200:

```
curl http://localhost:18000/v2/health/ready
```

However, a first request failed because the model repository configured `default_speaker=eng`, but no corresponding speaker embedding existed:

`speaker_embeddings/eng.pt`

The server error was:

EasyMagpieTTS: no speaker embedding .../speaker_embeddings/eng.pt for speaker_id 'eng'

This confirmed that Triton and vLLM loading were working, and that the remaining functional blocker was the missing speaker embedding.

Baking the Default English Speaker Embedding

The repository contained context audio samples under:

`context_audios/audio_context_samples/`

For the default English speaker, I used:

`context_audios/audio_context_samples/english_audio_context_samples/Emma_Additional.flac`

This file was a mono 22.05 kHz FLAC, about 6.98 seconds long. The conversion path uses a 5 second context window by default, so this was suitable.

Rather than rerunning the full model conversion and rewriting the 2.6 GB `model.safetensors`, I reused the conversion script's speaker extraction function directly:

```
extract_speaker_embedding(model, context_audio_path,
context_audio_duration)
```

The host Python environment did not have all NeMo conversion dependencies, so I used a disposable container based on the existing Riva/NeMo image:

```
gitlab-master.nvidia.com:5005/dl/riva/riva-speech/
riva:dev..siddhartht..magpie_may26_2605.54159250-linux-x86_64
```

That image had most of the NeMo stack, but needed temporary in-container installs of:

```
loguru
causal-conv1d
mamba-ssm
```

Those dependencies were installed only inside the disposable extraction container. They were not installed into the active host NeMo development environment.

The extraction loaded:

- the EasyMagpieTTS .nemo checkpoint
- the spectral codec .nemo
- the phoneme tokenizer
- the English context audio

It generated:

```
examples/tts/easymagpie_vllm_omni/easymp_vllm_model/  
speaker_embeddings/eng.pt
```

The saved speaker embedding had:

```
shape = (64, 1536)  
dtype = float32  
source audio =  
context_audios/audio_context_samples/english_audio_context_samples/  
Emma_Additional.flac  
checkpoint = 2605_EMTTS_SmallMamba_Step150k_posttrained_epoch12
```

I then corrected file ownership so the generated eng.pt belonged to the workspace user rather than container root.

Final Triton Validation

After baking eng.pt, I restarted Triton with the same command and waited for the model repository to load.

The logs showed:

```
codec    READY  
easymp   READY  
EasyMagpie initialized (default_speaker=eng, codec_noop=False)
```

The health endpoint returned HTTP 200 again.

End-to-End Request

I ran one whole-text gRPC request through the existing benchmark client:

```
docker exec easymp-triton-codex bash -lc \  
  'printf "utt1\tHello world.\n" > /tmp/easymp_one.txt && \  
  python3  
examples/tts/easymagpie_vllm_omni/scripts/benchmark_service.py \  
  --text-file /tmp/easymp_one.txt \  
  --triton-url localhost:8001 \  
  --no-warmup \  
  -n 1 \  
  -c 1 \  
  --verbose \  
'
```

```
--chunk-timeout 120 \  
--output-dir /tmp/easymp_e2e_out'
```

The request succeeded:

1 ok / 0 failed

The benchmark reported:

0.88s audio in 25.63s wall time
TTFA 25590.7 ms
ITL 11.0 ms
5 audio chunks
0 underruns

The output audio was copied to the host:

/tmp/easymp_e2e_out/utt1.wav

Audio properties:

channels: 1
sample rate: 22050 Hz
precision: 16-bit PCM
duration: 0.88 seconds

The server log confirmed the request used the baked default speaker:

speaker=eng text='Hello world.'

RTFX Observation

For the cold first request:

$RTFX = \text{audio duration} / \text{wall time} = 0.88 / 25.65 = \text{about } 0.03x$

This cold result was dominated by first-request time to first audio:

TTFA = about 25.59 seconds

The streamed chunks after first audio were much faster:

playback/fetch factor = about 18.5x realtime

A meaningful steady-state RTFX measurement should be taken with warmup enabled and multiple requests.

Cleanup

After validation, I stopped the Triton container:

```
docker stop easymp-triton-codex
```

GPU memory returned to idle display-only usage.

Final State

Completed:

- Triton 26.02 base image pulled.
- Custom easymp-vllm-omni runtime image built.
- Codec TensorRT plan generated from codec.onnx.
- Default English speaker embedding eng.pt baked.
- Triton loaded both codec and easymp successfully.
- End-to-end gRPC request returned valid audio.

Large generated artifacts remain untracked and were not committed:

```
context_audios/  
examples/tts/easymagpie_vllm_omni/codec.onnx  
examples/tts/easymagpie_vllm_omni/easymp_vllm_model/  
examples/tts/easymagpie_vllm_omni/model_repository/codec/1/
```

No vLLM installation was performed in the active host NeMo development environment.